



SSE-208 计算机网络实验

网络编程

(UDP & TCP & Web 服务器 & UDP ping 实现)

2026.05

应用层 Application layer: overview



- 应用层协议原理 Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS 域名解析
- P2P applications
- video streaming and content distribution networks 视频流和内容分发网
- socket programming with UDP and TCP 套接字编程



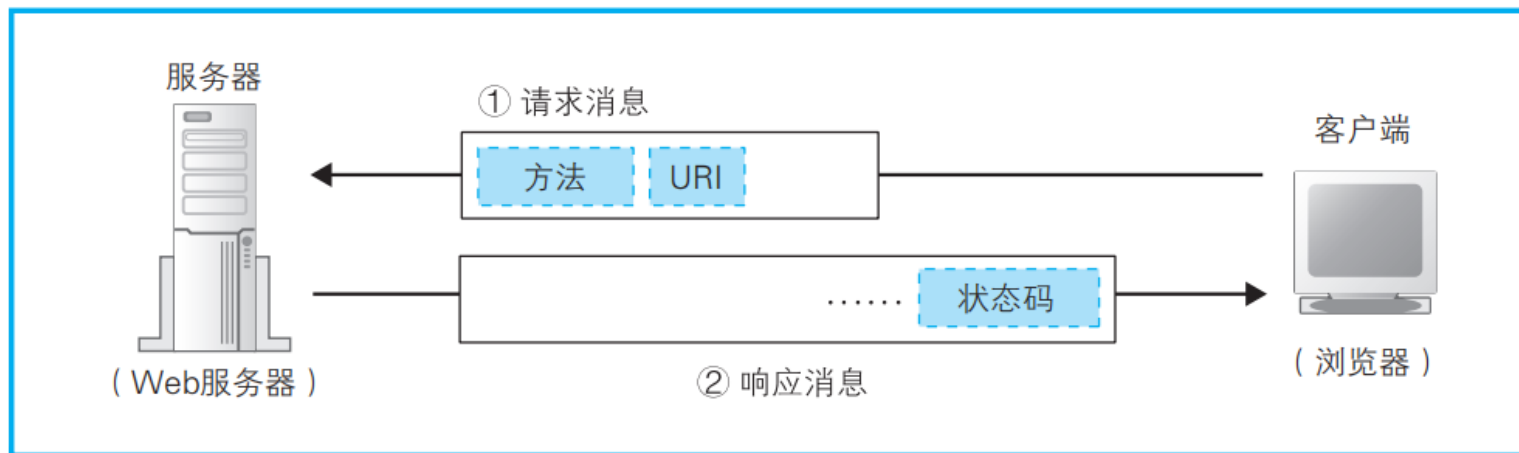
从输入网址到网页显示，期间发生了什么



总体流程

- 1. 用户Web浏览器解析 URL，确定目标服务器和文件名
- 2. 浏览器根据这些信息来生成 HTTP 请求消息
- 3. 浏览器查询本地HTTP缓存 (强制缓存, 协商缓存)
- 4. 浏览器委托操作系统向DNS 服务器查询目标服务器的 IP 地址

从浏览器中输入网址开始



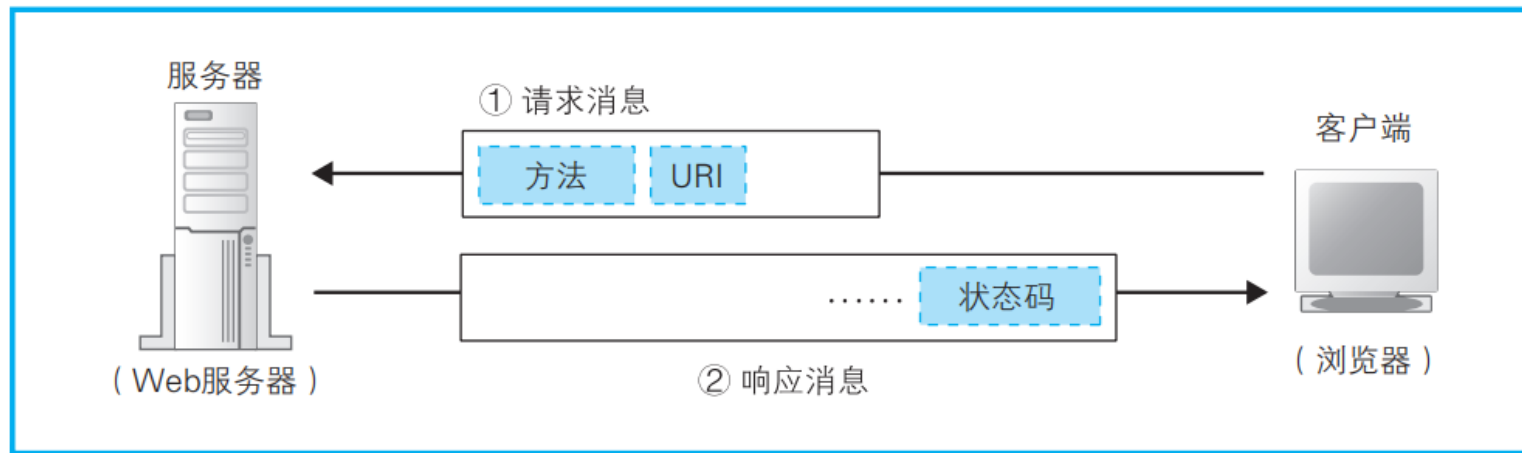
浏览器生成消息



总体流程

- 5. 客户端浏览器向Web服务器发起http请求
- 6. Web服务器返回响应消息
- 7. 浏览器将数据提取出来并显示在屏幕上

中间发生了什么？

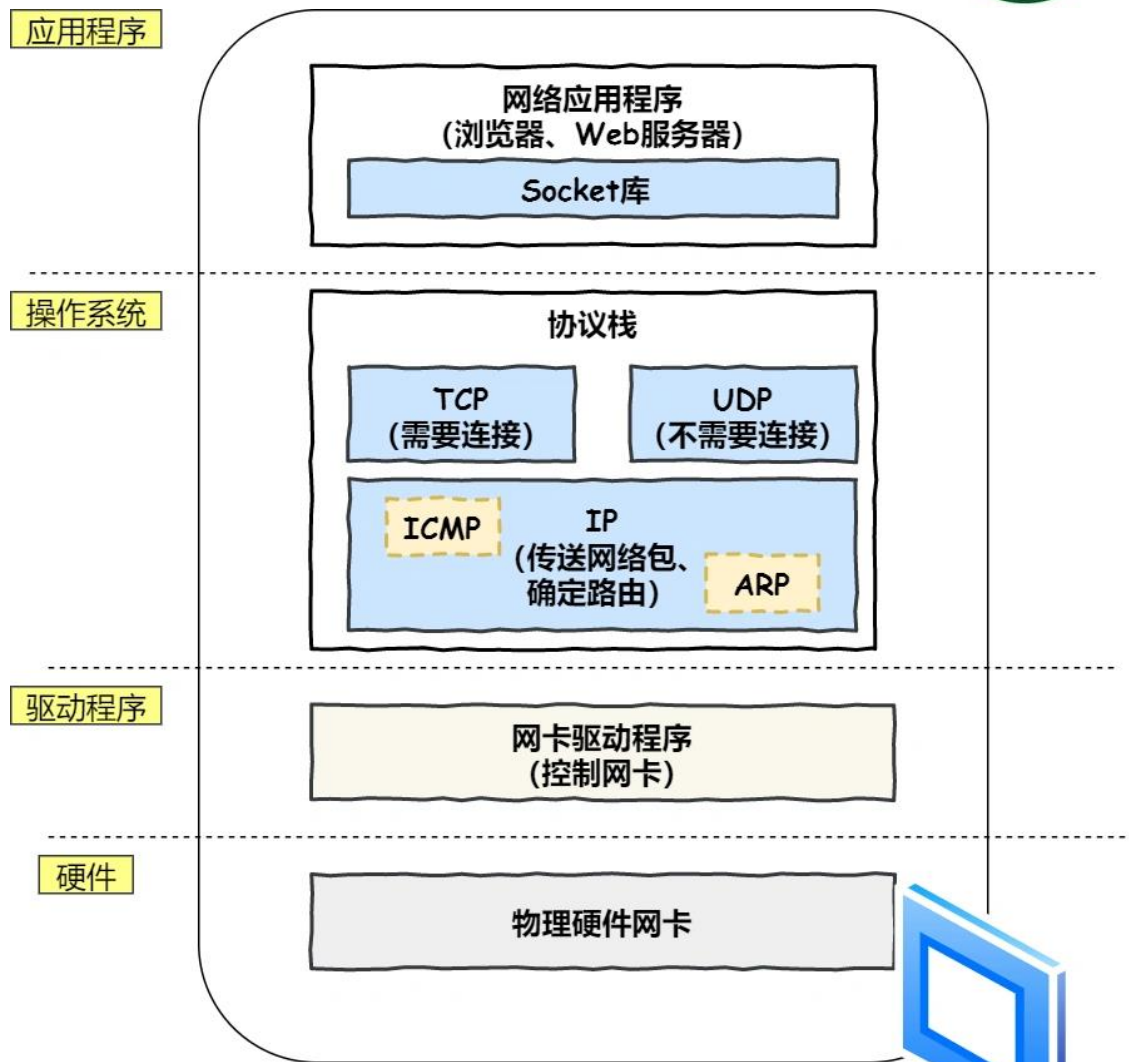


浏览器生成消息



■ 系统结构

- 应用层：应用程序（浏览器）调用 **Socket 库**，来委托协议栈工作。
- 传输层：接受应用层的委托执行收发数据操作
- 网络层：控制网络包收发操作
- 网卡驱动程序负责控制网卡硬件
- 网卡：数字信息转换为电或光信号在网线上传输

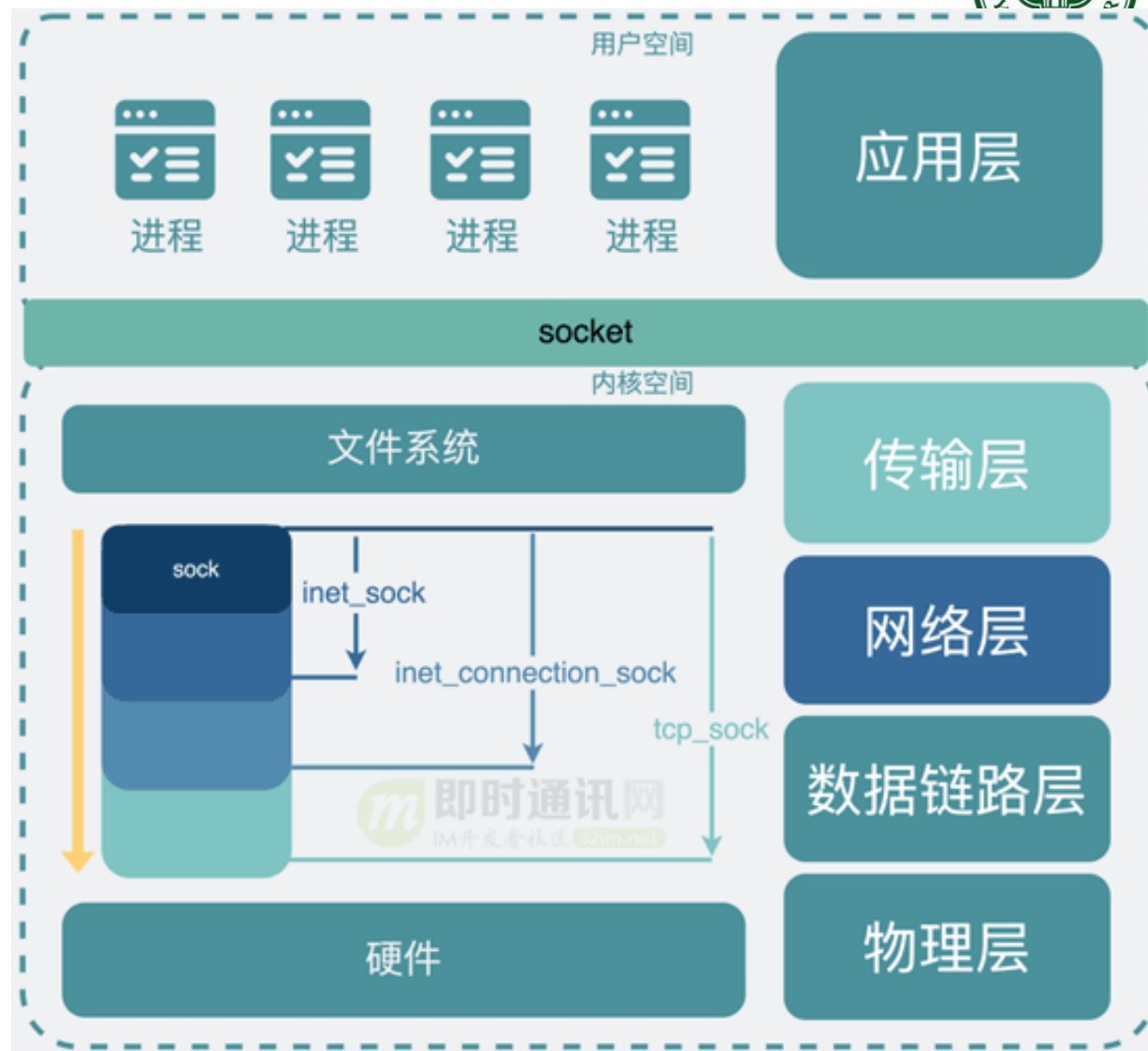


Socket 套接字介绍



Sock与Socket

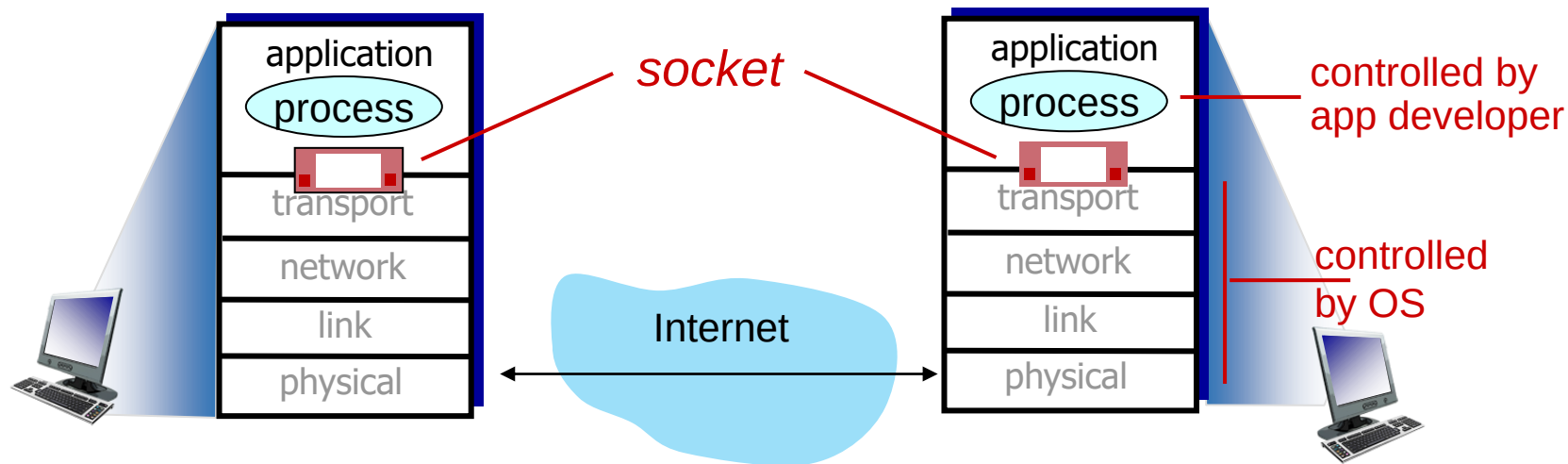
- 用户空间: **Socket** 提供了一些高度封装的接口, 便于应用程序使用网络传输功能
- 内核空间: 基于不同的协议和应用场景, **Sock**结合硬件, 共同实现了网络传输功能



Socket programming 套接字编程



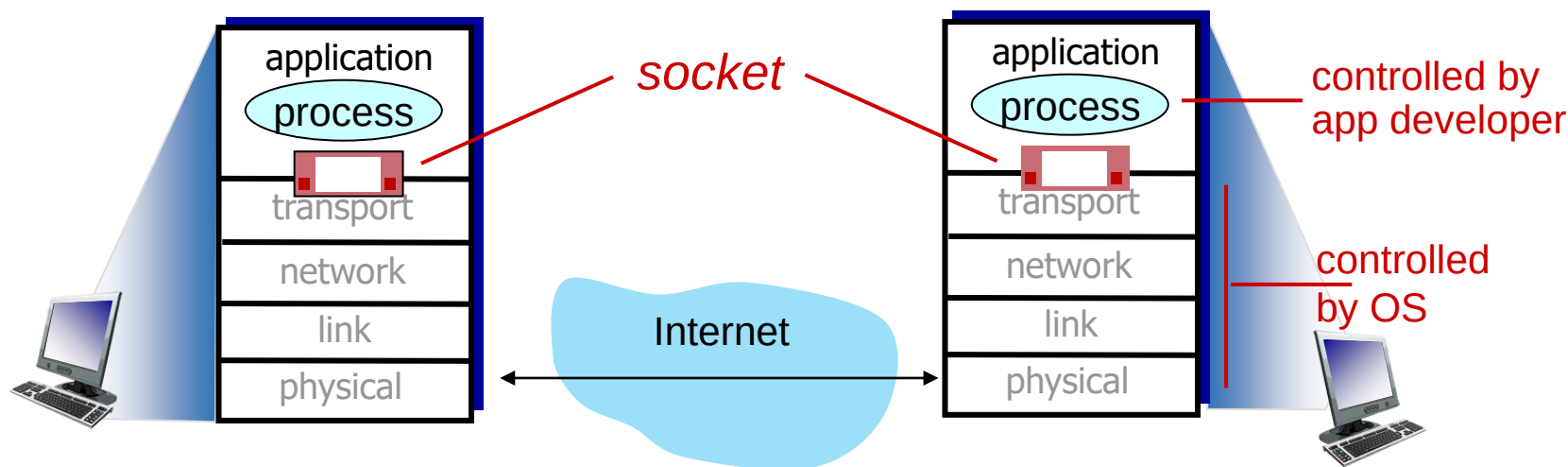
- 应用进程使用传输层提供的服务才能够交换报文，实现应用协议
- 对于TCP/IP协议栈：应用进程使用Socket API访问传输服务



Socket programming 套接字编程



- Goal: 学习如何借助sockets通信，构建C/S应用程序
- Socket: 分布式应用进程之间的门，传输层协议提供的端到端服务接口





2种传输层服务的socket类型:

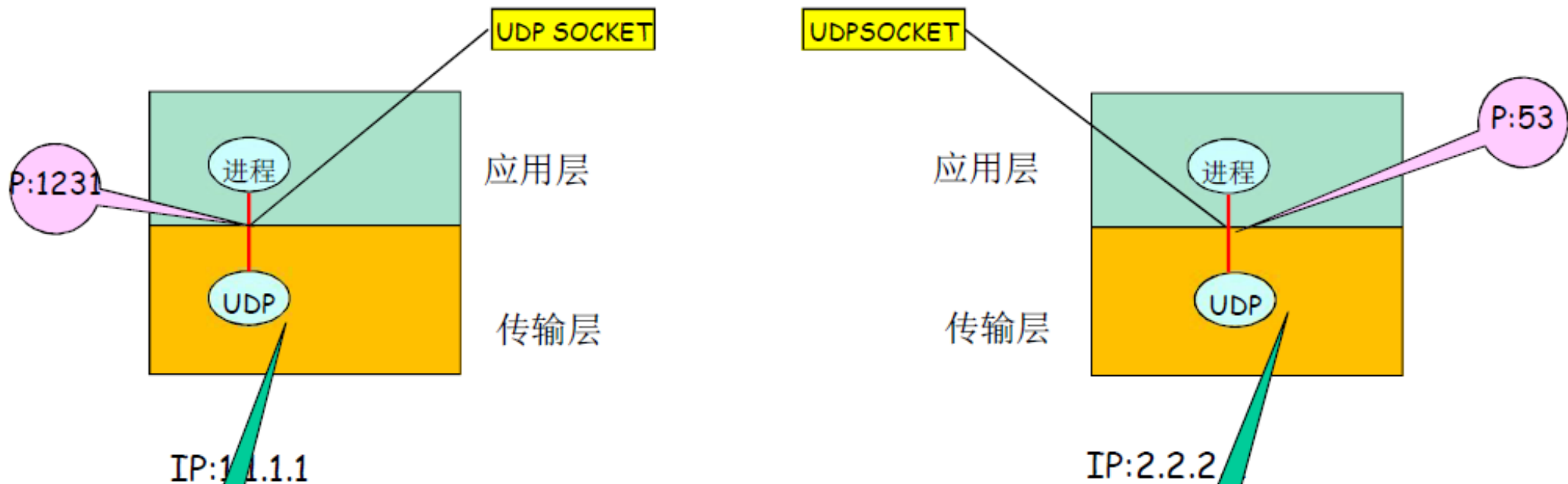
- **UDP:** 不可靠数据传输服务
- **TCP:** 可靠的, 字节流的服务
 - 不出错、不重复、不丢失、不失序
 - 流的概念: 传输层没有报文界限, 需要应用层自己维护单独的报文长度。

UDP Socket



- UDP服务，两个进程之间，通信前无需建立连接
 - 每个报文均独立传输
 - 前后报文可能发向不同的分布式进程（不同目标主机、应用）
- 只用一个整数表示**本应用**实体的标识
 - 报文可能传给另外一个分布式进程
- UDP socket 二元组：【**目标IP地址, 目标端口号**】
 - 传输报文时： 需要提供对方的IP以及端口
 - 接收报文时： 传输层需要上传对方的IP, port

UDP Socket



UDP socket : 用于指明应用进程所在端节点的本地标示

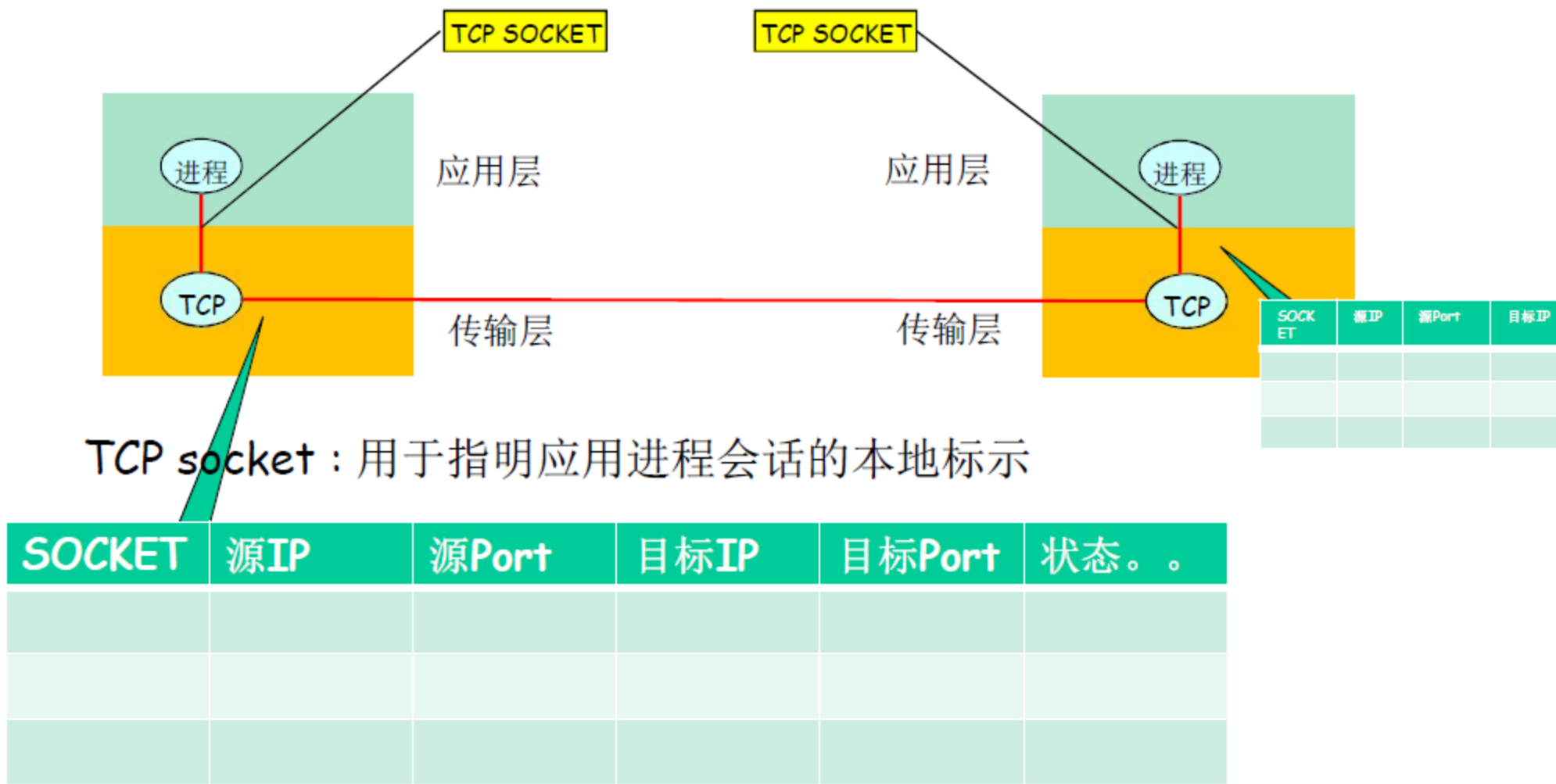
SOCKET	IP	Port	

SOCKET	IP	Port	

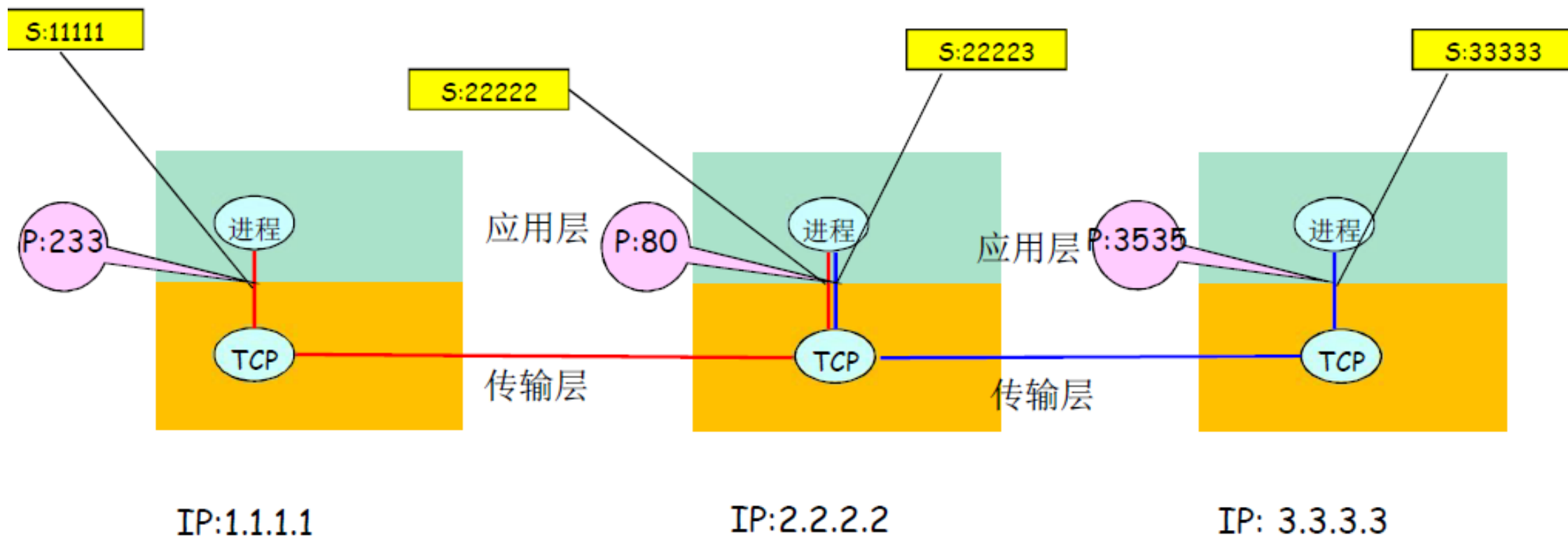
- 对于使用面向连接服务（TCP）的应用而言，套接字是4元组的一个具有本地意义的标识
 - 四元组：【源IP，源端口，目标IP，目标端口】
 - 唯一的指定了一个会话（2个进程之间的会话关系）
 - 应用使用这个标识，与远程的应用进程通信

- 不必在每一个报文的发送都要指定这4元组，使用Socket对象实例来代替。
如Socket **socket1** = new Socket()
 - 简单，便于管理

TCP Socket



TCP Socket

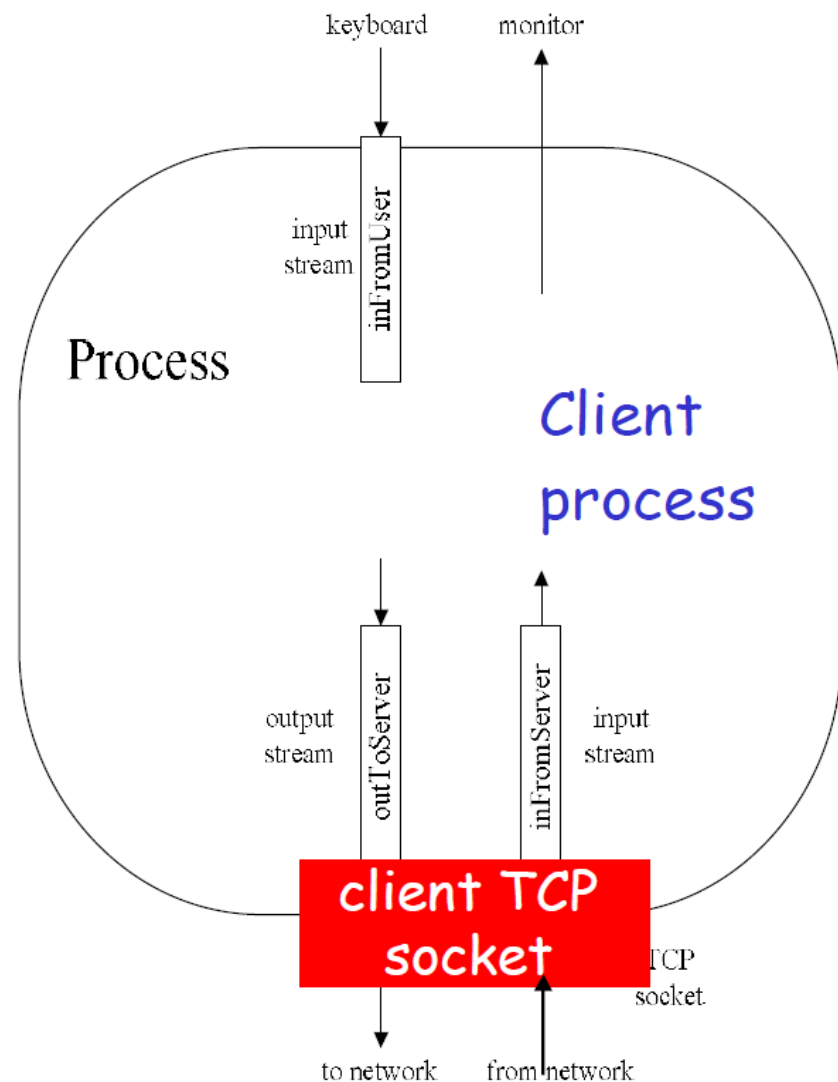


Socket programming



■ Application Example:

- 1. 客户端从标准输入装置读取一行字符，发送给服务器
- 2. 服务器从socket读取字符
- 3. 服务器将字符转换成大写，然后返回给客户端
- 4. 客户端从socket中读取一行字符，然后打印出来



Socket programming with UDP



UDP: 在客户端和服务端之间没有连接:

- 没有握手
- 发送端在每一个报文中明确地指定目标IP地址和端口号
- 服务器必须从收到的分组中提取出发送端的IP地址和端口号

UDP: 传送的数据可能乱序，也可能丢失

Client/server socket 交互: UDP



server (running on serverIP)

create socket, port = x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Example app: UDP client



Python UDPClient

包括Python的socket库

→ `from socket import *`

`serverName = 'hostname'`

`serverPort = 12000`

创建连接服务器的UDP socket

→ `clientSocket = socket(AF_INET,
SOCK_DGRAM)`

获取用户键盘输入

→ `message = raw_input('Input lowercase sentence:')`

将服务器名、端口附加到消息;发送到插座

→ `clientSocket.sendto(message.encode(),
(serverName, serverPort))`

将应答字符从套接字读入字符串

→ `modifiedMessage, serverAddress =
clientSocket.recvfrom(2048)`

打印收到的字符串并关闭套接字

→ `print modifiedMessage.decode()
clientSocket.close()`

Example app: UDP server



Python UDPServer

	from socket import *
	serverPort = 12000
创建UDP套接字	→ serverSocket = socket(AF_INET, SOCK_DGRAM)
绑定套接字到本地端口号12000	→ serverSocket. bind ("", serverPort))
	print (<i>"The server is ready to receive"</i>)
永远循环	→ while True:
从UDP套接字读取消息，获取客户端地址(客户端IP和端口)	→ message, clientAddress = serverSocket.recvfrom(2048)
	modifiedMessage = message.decode().upper()
将大写字符串发送回此客户端	→ serverSocket.sendto(modifiedMessage.encode(), clientAddress)

Socket programming with TCP



服务器进程必须先处于运行状态

- 创建欢迎socket
- 与本地端口**捆绑**
- 在**欢迎端口**上阻塞式**等待**接收用户的连接

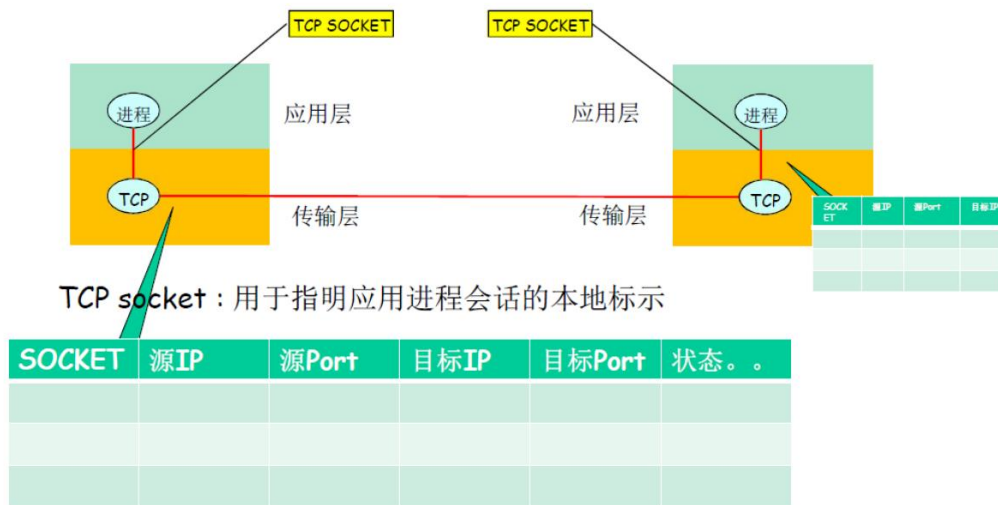
客户端主动和服务端建立连接

- 创建客户端本地套接字（隐式捆绑到本地端口）
- 指定服务器的IP，端口号，与服务端进行连接
- **当客户端创建套接字时**: 客户端TCP与服务端TCP建立连接

Socket programming with TCP



- 当客户端的连接请求到来时
- 服务器**接受**来自用户端的请求，解除阻塞式等待，返回一个**新的socket**（与欢迎socket不一样），与客户端通信
 - 允许服务器与多个客户端通信
 - 使用**源IP和源端口**来区分不同的客户端

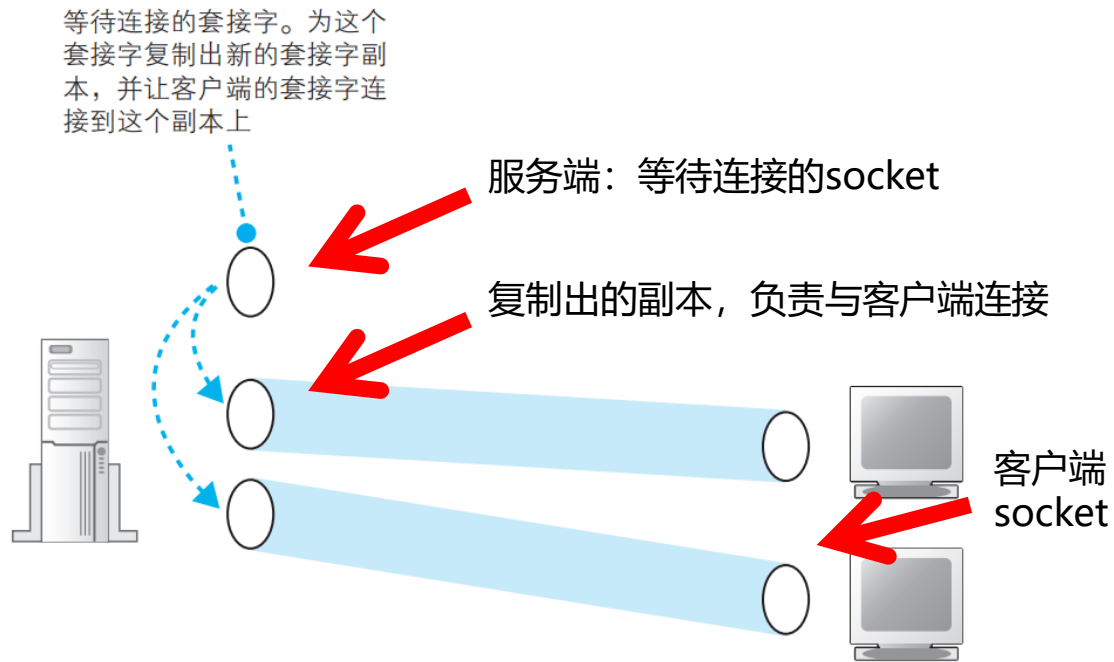


Application viewpoint
TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Socket programming with TCP



- 如果不创建新副本，而是直接让客户端连接到等待连接的 socket 上，那么就没有 socket 在继续等待新的连接了

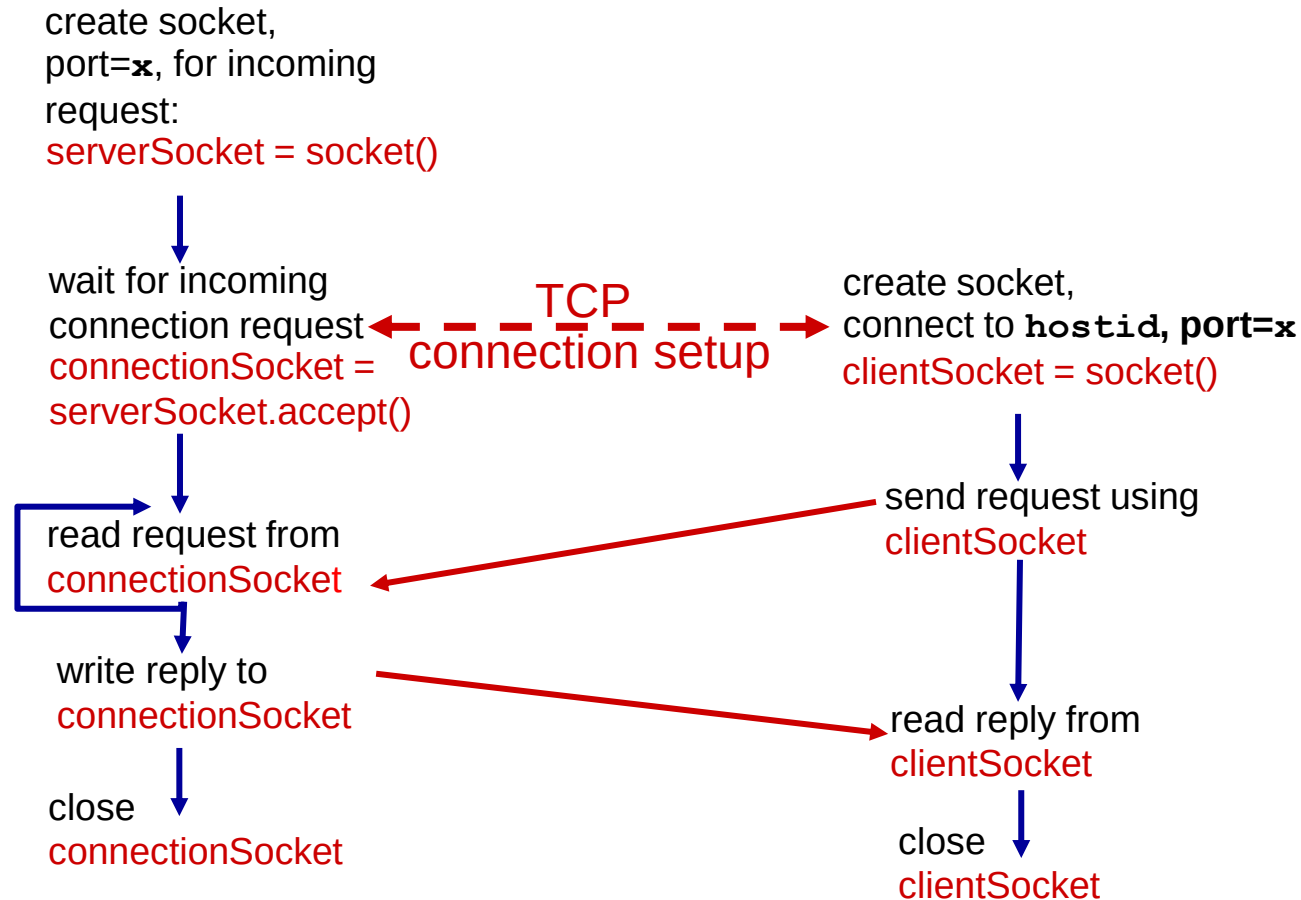


Client/server socket interaction: TCP



server (running on `hostid`)

client



Example app: TCP server



Python TCP Server

```
from socket import *  
serverPort = 12000
```

创建TCP欢迎套接字



```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))
```

服务器开始监听传入的TCP请求



```
serverSocket.listen(1)
```

```
print 'The server is ready to receive'
```

loop forever



```
while True:
```

服务器等待accept()传入请求, 返回时
创建新的套接字



```
    connectionSocket, addr = serverSocket.accept()
```

从套接字读取字节(但不是UDP中的地址)



```
    sentence = connectionSocket.recv(1024).decode()  
    capitalizedSentence = sentence.upper()  
    connectionSocket.send(capitalizedSentence.  
                           encode())
```

关闭到此客户端的连接(但不是欢迎套接字)



```
    connectionSocket.close()
```

Example app: TCP client



Python TCPClient

为服务器创建TCP套接字，远程端口为12000



```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

无需附加服务器名、端口



对比 UDP Client:

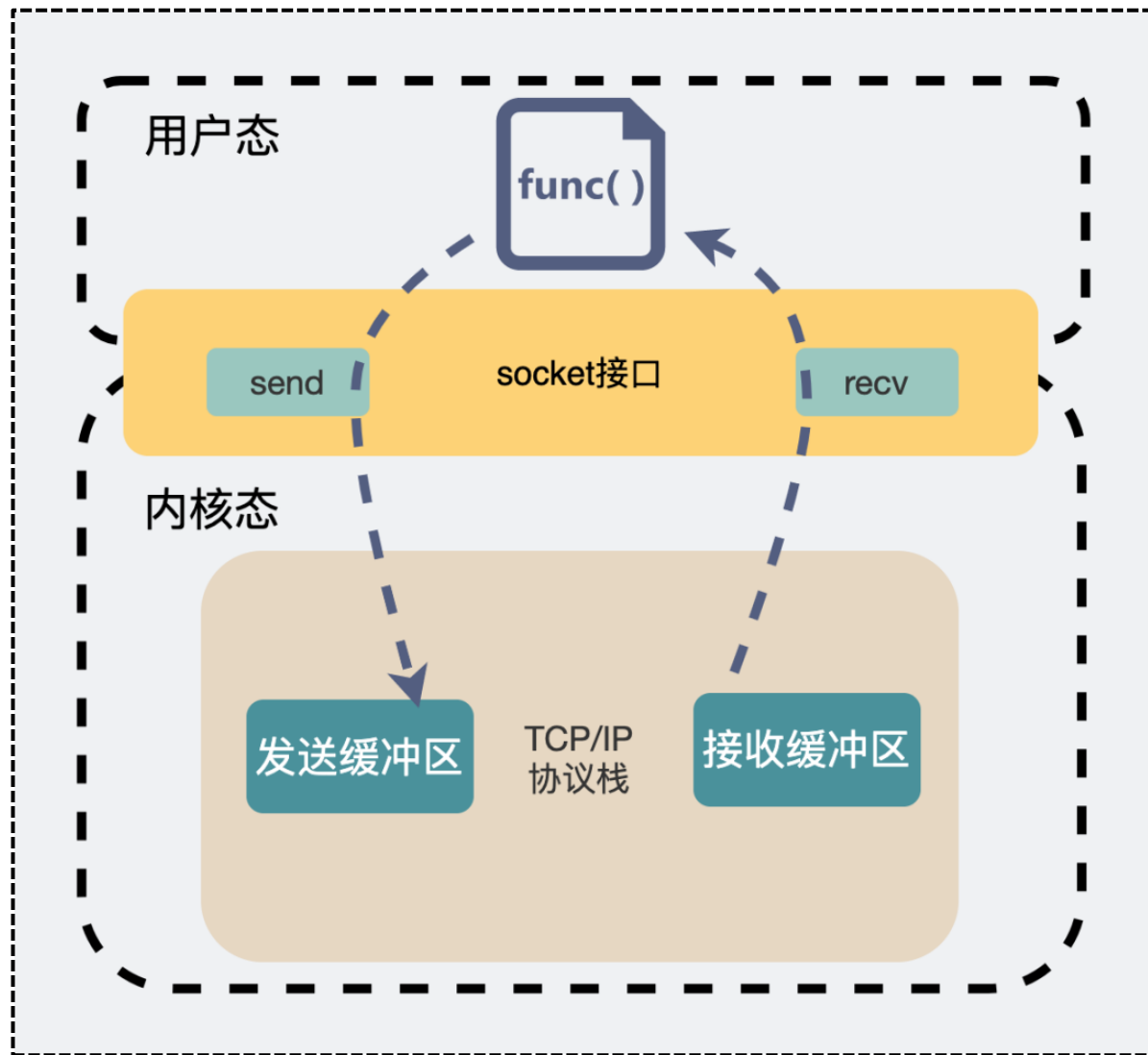
```
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
```

Socket 缓冲区



■ Socket 数据收发过程

- 每个 socket 被创建后，无论使用的是 TCP 还是 UDP 协议，都会创建自己的输入缓冲区和输出缓冲区



■ Socket 数据收发过程 -- send (阻塞模式下)

- 发送缓冲区可用长度 vs. 待发送数据长度
 - 发送缓冲区可用长度 > 待发送数据长度, 则数据将全部被拷贝到发送缓冲区
 - 发送缓冲区可用长度 < 待发送数据长度, 则数据能拷贝多少就先拷贝多少 (分批拷贝), 一直等待, 直到数据可以全部被拷贝到发送缓冲区为止, 才调用返回。
- `send()` 并不能保证数据已经发送到对端, 仅仅是把应用层 buffer 的数据拷贝进 socket 发送缓冲区中,
- 至于缓冲区的数据什么时候发, 发多少, **取决于发送窗口、拥塞窗口以及当前发送缓冲区的大小等条件**
- `send()` 可能只发送部分字节, `sendall()` 会循环发送直到完成或出错

■ Socket 数据收发过程 -- recv (阻塞模式下)

- 当接收缓冲区没数据时，程序就会一直阻塞等待，直到有数据可读为止
- `socket.recv(bufsize[, flags])`, `bufsize` 指定一次接收的**最大数据量**
 - 如果对方发送了超过`bufsize`的数据，在这种情况下要调用几次`recv`函数才能把套接字接收缓冲区中的数据拷贝完
 - `recv()` 所做的工作，仅仅是把`socket`接收缓冲区的数据拷贝到应用层 `buffer` 里面
- TCP 协议会保证数据的有序完整的传输，但是如何去正确完整的处理每一条信息，是开发人员的事情。